Version Control: Git and GitHub

Andrea Moglia, Marco Paracchini

Laboratory of Nearable Technologies for Health Data Science
A.Y. 2025/2026

Git: Version Control System

Git is a **distributed version control** and source code management system. It is a free software that allows *fast and easy* project management.

A **Version Control System** (VCS) is a software that helps developers to work together in teams and maintain an *ordered and complete* history of their work.

Git: a Time Machine

A version control software, like Git, allows developers to maintain a history of source files, allowing to **revert unwanted changes** and restore the code to a previous *version* in case of errors or failure.

In this case, Git works as a sort of **time machine** that keeps track of our code

Git: Why?

Well, to avoid *this*…

Nome	Tipo
main.c	File C
main_old.c	File C
main_working.c	File C
main_working_FINAL.c	File C
user_lib.c	File C
user_lib.h	C Header File
user_lib_DO_NOT_TOUCH_THIS.c	File C
user_lib_rev0.h	C Header File
user_lib_rev1.h	C Header File

… and have something like *this* instead.

Nome	Tipo
main.c	File C
user_lib.c	File C
user_lib.h	C Header File

Git: Version Control System

Git is a command line tool

Most modern code editors/ IDE (e.g., Visual Studio Code) have a built-in Git client that makes it easy to interact with Git using graphical user interface.

However!

It is still a good idea to learn how to use Git from the command line:

- It gives you a better understanding of how Git works;
- It allows you to use Git on any computer, even if it doesn't have your favorite dev tools installed;
- It allows you to use Git on a remote server, that doesn't gave any graphical user interface.

Git: Terminology

Repository

A “repo" is a storage space where your project lives. It can be local storage on your computer, or it can be a remote storage on a service like GitHub.

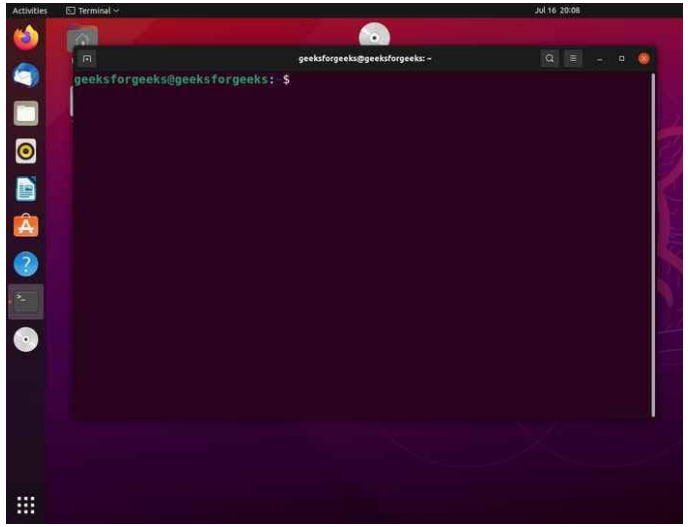
Commit

A snapshot (or milestone) of your repository at a certain point in time. A commit is created to capture the state of a project at that point of time. It only carries information about the changes that were made to the repository since the last commit.

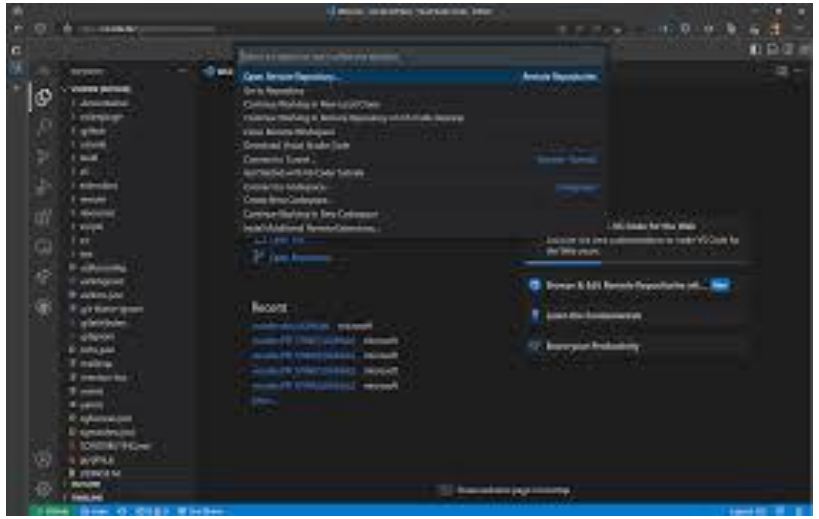
Branch

A parallel version of a repository. It allows you to work on a new feature of your project without affecting the main version of the project. Once you are done working on the new feature, you can merge the branch back into the main version of the project.

Ruunning Git



Terminal



Visual Studio Code

Git: Configuration

The first time you use Git on your machine, you have to do a quick-and-easy configuration step, to setup your name and e-mail.

Open the Git BASH and type the following commands

git config --global user.name "Your Name"
git config --global user.email "your_email@example.com"

To check:

git config --global user.name

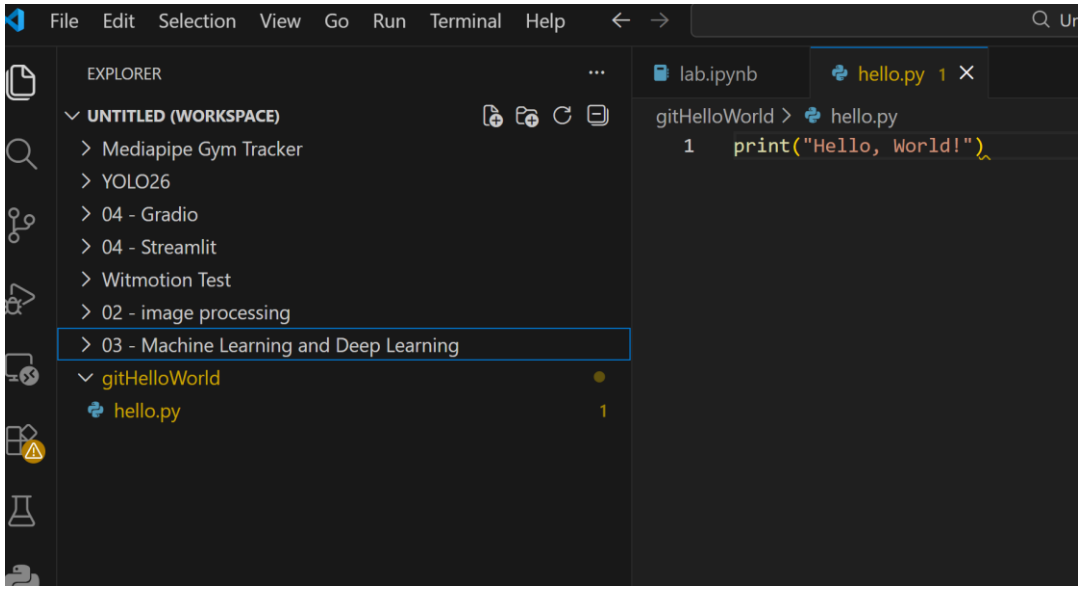
These are not credentials for GitHub: they serve as identifiers for the authors of any change to the tracked files.

Git: Basic Commands

man git	Manual for git
git help	Help for git
git init	Init a new repository in the local machine
git clone	Clone either a local or (most-likely) remote repo
git add	Add an untracked file or stage a modified file
git status	Shows modified/new/untracked files
git commit	Creates a commit of the staged files
git tag	Assign a tag to a commit
git rm	Remove a tracked file
git log	Shows the commits history
git diff	Shows the difference between two *versions* of a file
git checkout	Switch branch or update/restore files in the working tree

Git: Hello, World!

1. Create a folder named gitHelloWorld
2. <u>Create a new local repository</u> with **git init**
3. Create a file hello.py that prints “Hello, World!”



Git: status, add, commit

4. Type **git status** to check the current status of your repo

5. Add the file hello.py from your working directory to the stage area: **git add hello.py** Without this step, Git won't know which changes you intend to include in your next commit. Git scans and records the changes you have made to hello.py in its staging area.

6. Type again **git status** to see how things changed

7. Commit the tracked files with **git commit –m "My first commit"**
   It transforms what's in the staging area into an immutable snapshot stored within Git's repository history. Each commit is given a unique hash (a cryptographic checksum) to identify it. This hash is basically an ID tied to that exact version of your files. It also saves a pointer to this commit in the repository. Git only commits what is staged, not everything you changed

4. Type again **git status** to see how things changed

Git: diff, log

Let us change the source code in hello.py

- You can check what files have been modified with **git status**
- **git diff <file_name>** after you change a file, <u>but before commit</u>
- **git log** to see all the changes
- Type “q” to exit
- If you have many files and want to see changes on a single file:
  git log <file_name>

Make a change to hello.py

- **git status**
- **git add hello.py** (it means:“stage this”)
- **git commit –m “Change message”** (“save the staged changes)
  Or if the file is already tracked/ committed before:
  git commit –am “Change message”
- You can compare different commits with

  git diff using the SHA-1_ID1 SHA-1_ID2

SHA-1 hash

```
commit 323fe5630c5c11087d7b412aa188739483d3f7c5 (HEAD -> master)
Author: Andrea <andmog77@gmail.com>
Date:   Wed Feb 5 09:30:57 2025 +0100

    Changed message

commit b2724343f48fc9c650f3ef7e51f63986ec765467
Author: Andrea <andmog77@gmail.com>
Date:   Wed Feb 5 09:07:05 2025 +0100

    My first commit
```

Git: the Three States

Modified: you have changed the file but have not committed it to your database yet;

Staged: you marked a modified file in its current version to go into your next commit snapshot. Git knows about the change, but it is not permanent in the repository;

Committed: the data is safely stored in your local database.

```
git add
git status
git log
```

```
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   file1.js
        new file:   src/file2.js

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   src/constants.ts
```

staged changes

unstaged changes

Git: checkout, restore

We can restore one or more unstaged files with:

git checkout <file_name>

Or:

git restore <file_name>

We can restore one or more staged files with:

git restore --staged <file_name> **(1)**

git restore <file_name> **(2)**

Git and GitHub

- **Git** alone is great to handle **local repositories**, i.e. for *solo developers* that work always on the same machine.
- Integration with **GitHub** allows to **work in a team** on a group project.
- When a new project is created, one member of the team has to <u>create a new repository on GitHub</u> and *link* it to the local repo on their machine.

  git remote add origin + url@GitHub

  git remote add origin Hyp
- Check if it worked

  git remote –v
- Push your code to GitHub

  git push -u origin master
- The other team members will clone the remote repo to their machines

  git clone + url@GitHub

GitHub: Hello, World!

1. To create a new repo on GitHub, simply log in and click on

2. Follow the quick setup to create the repo (do not add a README yet)
3. Now copy the URL of your repo and append .git to it

   e.g.: **https://github.com/*[your_username]*/gitHelloWorld.git**

4. Now we can *transfer* the local repository to the remote server

   git remote add origin <remote_repo_URL.git>

   git remote –v

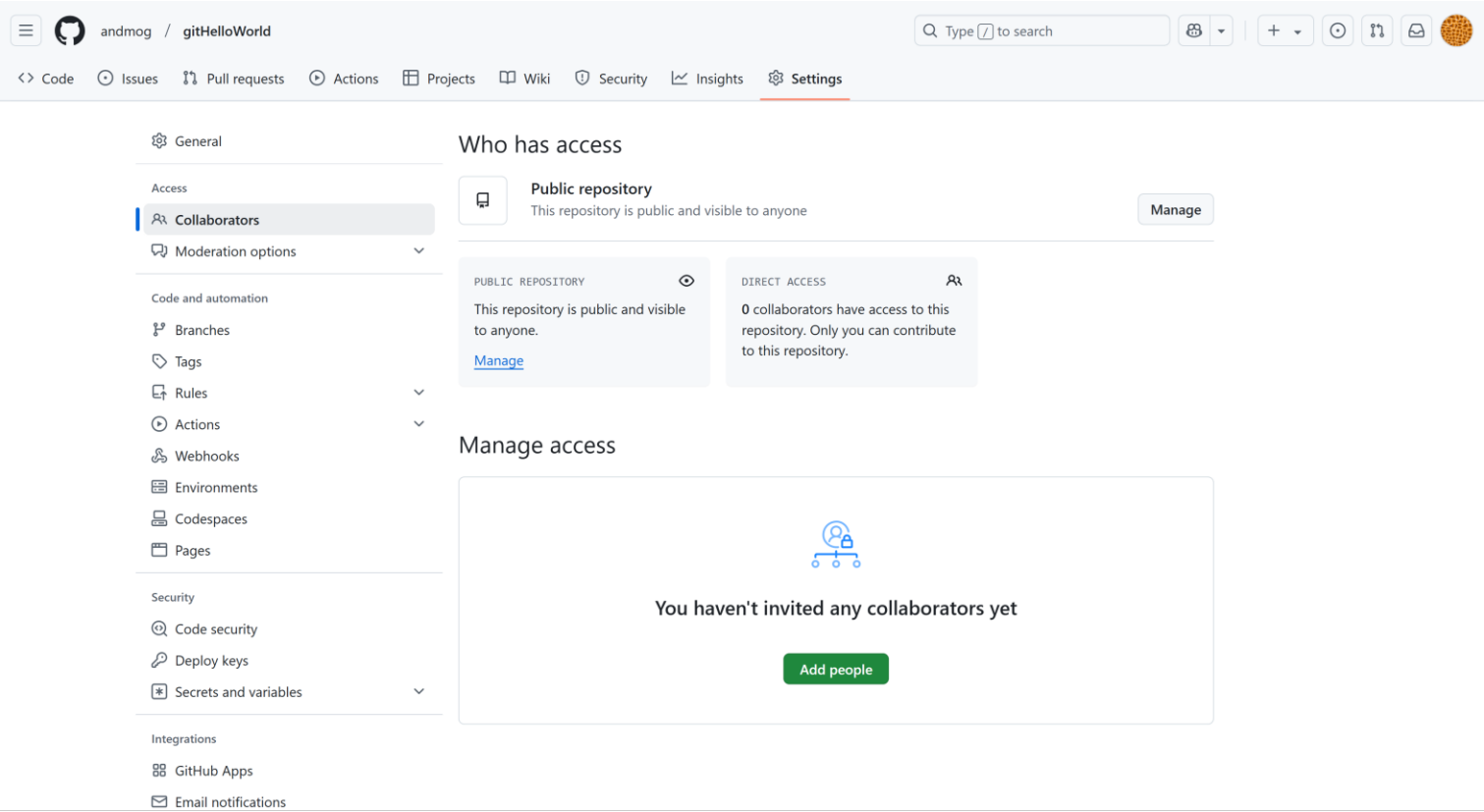
   git push –u origin master

5. To clone this remote repo on another machine, we can use the **clone** command. This will create a folder named as the remote repo (if not specified otherwise with the option <folder_name>)

   git clone <remote_repo_URL.git> [<folder_name>]

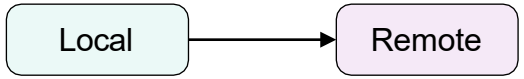
GitHub – Hello, World!

6. Finally, the owner of the repo on GitHub can add the other team members as collaborators in the **Settings** menu.



Git: Push and Pull

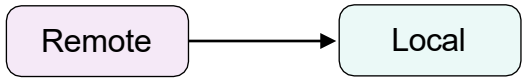
Push



The command **git push** is used to *synchronize* a commit on a local machine with the remote repository. Pushing makes local changes available on the remote server to all the team members.

git push origin master

Pull

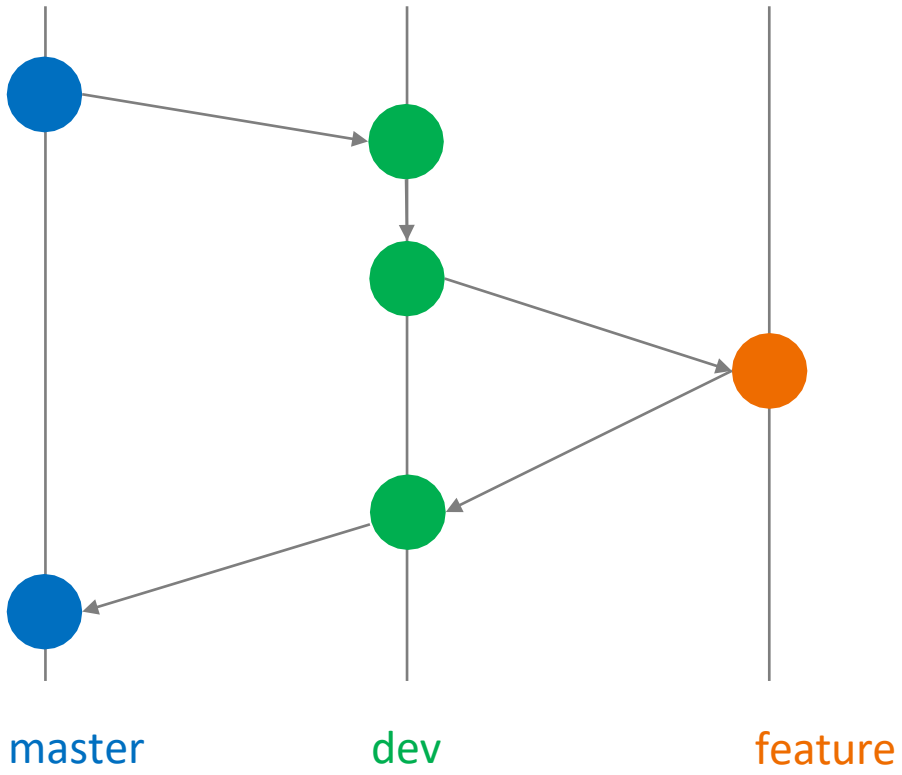


The command **git pull** is used to simultaneously fetch and merge. This means that pulling downloads (fetch) new commits previously pushed to the remote repository and updates the local repository (merge).

git pull origin master

Branching

Branching is one of the most powerful and useful features of git. It allows you to *freeze* the source code of a project and work on an exact copy of it.



Branching

When you init a repo, the default branch master is created for you automatically. Typically, here you have the working source code of the project.

Additional branches can be created for instance for specific features (feature) to be added to the stable branch (master), but separate from a development branch (dev).

Once we are satisfied with the new code, we can **merge** it back to master.

While it is not mandatory to create branches, it is highly recommended to keep a project well organized.

Branching

To create a new branch:

git branch <branch_name>

git checkout –b <branch_name> (in this case, we also *switch* to the new branch right after creating it)

git switch –c <branch_name> (in this case, we also *switch* to the new branch right after creating it)

To list all branches:

git branch

To view all branches:

git log --all --graph

To switch to one branch:

git switch master

To merge two branches:

git merge <other_branch> to merge one branch <u>into the current</u> <u>branch</u> (e.g. merge dev into master).

Git: Best Practices

- ✓ **Commit frequently**: Commit small units of work often. This makes it easier to understand what changed and why.
- ✓ **Write descriptive commit messages**: Good commit messages make it much easier to navigate project history and troubleshoot issues.
- ✓ **Use feature branches**: Instead of working directly on **main** branch, create separate branches for new features, then merge them back when ready.
- ✓ **Check your status often**: Use **git status** to see what's changed and what's staged. This helps you stay organized and ensures you don't commit unintended files.

GitHub: Fork

- A fork is a copy of an entire repository that allows you to freely experiment with changes without affecting the original project.
- Forking is often used in open-source projects to allow contributors to make modifications before proposing to merge these changes back into the main project.
- When you fork a repository, you make an entirely separate copy of the project under your own account. This is useful when you do not have write access to the original repository and want to propose changes to it.

Fork and Branch: Differences

- **Scope of isolation**: Branches are isolated within the same repository, allowing easier collaboration among team members who have access to the repository. Forks, on the other hand, create a completely separate copy of the repository, which is useful for outside contributors.
- **Access and permissions**: Branching is suitable when team members have write access to a repository and need to implement or experiment with features independently. Forking is preferable when contributors do not have direct write access to the project's repository.
- **Integration process**: Changes from a branch are usually merged back into the main branch through a pull request within the same repository. In contrast, changes from a fork require a pull request to the original external repository.

Fork and Branch: When to use them

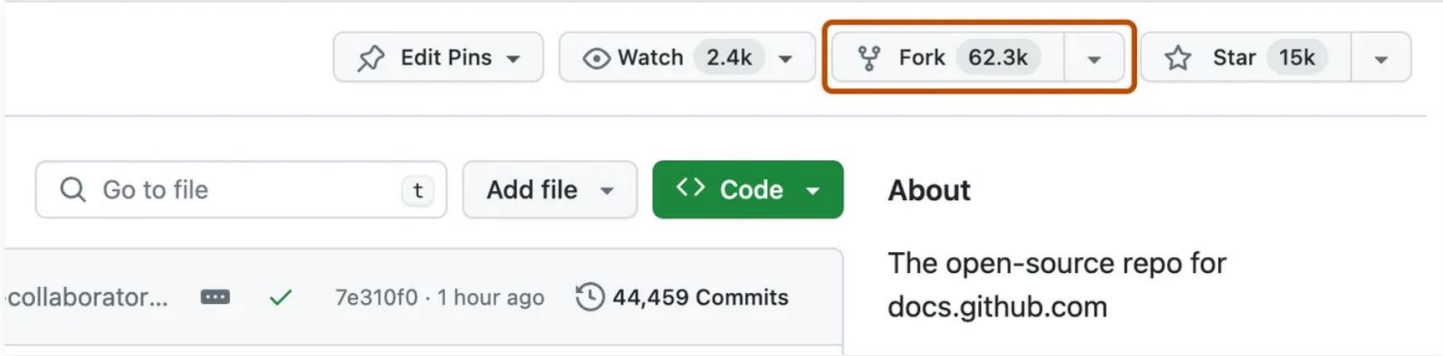
Use a branch when:

- You have write access to the repository.
- You want to make quick fixes or develop features collaboratively within the same project.
- You aim to frequently synchronize your code with the main project.

Use a fork when:

- You do not have write access to the repository and want to suggest changes.
- You want to maintain a separate project based on the original project or use it as a starting point.
- You are contributing to an open-source project and need to handle your changes independently before merging them.

GitHub: Fork



1. Under "Owner," select the dropdown menu and click an owner for the forked repository.
2. By default, forks are named the same as their upstream repositories. Optionally, to further distinguish your fork, in the "Repository name" field, type a name.
3. Optionally, in the "Description" field, type a description of your fork.
4. Optionally, select **Copy the DEFAULT branch only.**
5. For many forking scenarios, such as contributing to open-source projects, you only need to copy the default branch. If you do not select this option, all branches will be copied into the new fork.
6. Click **Create fork.**

References and Useful Resources

- Git documentation https://git-scm.com/docs
- https://github.com
- Bash shell: https://en.wikipedia.org/wiki/Bash_(Unix_shell)
- https://jonas.github.io/tig/
- Windows Terminal: https://github.com/microsoft/terminal
- Git Video Tutorials: https://git-scm.com/videos
- Atom: https://atom.io/
- https://www.youtube.com/watch?v=XX-Kct0PfFc